Lab 9: Seven-Segment Displays and Code Converters

Complete Prelab and Postlab Derivations

Name: _____ Student ID: _____

Course: CPE100L Experiment date: _____

Karnaugh maps – Boolean minimization – Circuit connections

Scope of this report. This English version contains the same design, derivations, connection plans, expected results, and postlab answers as the Chinese version, with exhaustive offline Boolean checks. Quartus compilation, ModelSim simulation, and DE0-CV board testing have not been performed. Expected results are identified separately from measurements. Design diagrams use LaTeX/TikZ; the appendix adds attributed external circuit illustrations. Board photographs, LE counts, actual time spent, and TA sign-off remain to be supplied.

Source and segment-pattern convention. The task requirements come from pages 4–7 of `CPE_100_Lab_9.pdf`; the operation-select codes are confirmed by pages 2–3 of the adjacent Lab 8 handout. On page 2 of Lab 9, the prose says that 6 omits segment a and 9 omits segment d, but Table 2 assigns 0 to both positions, which lights them. This report consistently follows the numeric entries in **Tables 2 and 3**: a is on for 6, and d is on for 9. If the TA instead requires the prose convention, update the truth table and repeat the minimization rather than mixing the two conventions.

Experiment 3 diagram ambiguity. The handout requests two `hex7seg` symbols and also an independent letter decoder for HEX1. The direct implementation here uses one `hex7seg` for the result on HEX0 and a separate letter decoder for HEX1, because lowercase o is not a standard hexadecimal glyph. Section 4.2 also gives a complete alternative that retains a second `hex7seg` and overrides its OR-mode glyph.

Contents

1 Notation, segment patterns, and minimization method **3**
1.1 Bit order and active-low outputs . . . 3
1.2 Complete target truth table . . . 3
1.3 How to read and simplify every K-map . . . 4

2 Prelab 1: BCD-to-seven-segment decoder **5**
2.1 BCD decoder: complete gate connections . . . 9
2.2 Sharing product terms between outputs . . . 9

3 Prelab 2: complete hexadecimal decoder **10**
3.1 hex7seg: complete gate connections . . . 14
3.2 Cost comparison . . . 15
3.3 Concrete gate circuit for hexadecimal segment a . . . 15
3.4 Theoretical timing for all sixteen codes . . . 16

4 Prelab 3: result and operation-letter displays **17**
4.1 Letter patterns and select logic . . . 17
4.2 Alternative retaining a second hex7seg . . . 17

5 Prelab 4: binary-to-Gray and BCD-to-excess-3 **18**
5.1 Deriving the four Gray-code equations . . . 18
5.2 Excess-3 truth table and bit-by-bit minimization . . . 18
5.3 Complete excess-3 connections . . . 21
5.4 Prelab question: common anode or common cathode? . . . 21

6 Postlab: operating principles and experiment connections **22**
6.1 Operating principles, time, and handout issues . . . 22
6.2 Global connection rules . . . 22
6.3 Experiment 1: displaying a three-bit number . . . 22
6.4 Experiment 2: complete hexadecimal display . . . 23
6.5 Experiment 3: two-bit ALU and two displays . . . 24
6.6 Experiment 4: two converters sharing one input . . . 26

7 Postlab: answers to the four questions **28**
7.1 Why does the two-bit sum fit one digit? What about a three-bit ALU? . . . 28
7.2 Which product term was shared, and how many gates were saved? . . . 28
7.3 Why can six extra hexadecimal rows increase decoder cost? . . . 28
7.4 Why are K-map rows and columns labeled in Gray order? . . . 28

8 Verification, conclusions, and remaining evidence **29**
8.1 Offline checks actually performed . . . 29
8.2 Design-stage conclusions . . . 29
8.3 Complete after actual experimentation . . . 29
8.4 Submission requirements and current status . . . 30

A Compilation and reproduction without Quartus **31**

B Source files and page references **31**

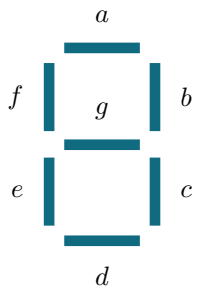
C Online circuit illustrations and source links **32**
C.1 Experiments 1 and 2: BCD and full hexadecimal decoders . . . 32
C.2 Experiment 3: ALU organization and display connections . . . 33
C.3 Experiment 4: correct four-bit binary-to-Gray circuit . . . 34
C.4 Experiment 4: BCD-to-excess-3 gate reference . . . 35

1 Notation, segment patterns, and minimization method

1.1 Bit order and active-low outputs

Define the display-decoder inputs as $D = d_3$, $C = d_2$, $B = d_1$, and $A = d_0$. D is the most significant bit and A is the least significant bit. These names are separate from the ALU operands a_1a_0 and b_1b_0.

Juxtaposition denotes AND, + denotes Boolean OR, an overbar denotes NOT, and $\oplus$ denotes XOR. Arithmetic addition is explicitly described as a sum or an ADD operation. The segment vector is always written in the order (a, b, c, d, e, f, g): **0 turns a segment on; 1 turns it off**.



$\text{HEX}x[0] = a,\ \text{HEX}x[1] = b$
$\text{HEX}x[2] = c,\ \text{HEX}x[3] = d$
$\text{HEX}x[4] = e,\ \text{HEX}x[5] = f$
$\text{HEX}x[6] = g$

For a bus written as $\text{HEX}x[6:0]$, the left-to-right bit string is $gfedcba$. Reverse the $abcdefg$ strings in this report when writing that bus value. The decimal point is not used.

1.2 Complete target truth table

The BCD decoder constrains only inputs 0–9; inputs 10–15 are X for all seven outputs. The hexadecimal decoder constrains all sixteen inputs and has no don't-cares.

Value	$DCBA$	Glyph	$abcdefg$	$gfedcba$	Valid BCD?
0	0000	0	0000001	1000000	Yes
1	0001	1	1001111	1111001	Yes
2	0010	2	0010010	0100100	Yes
3	0011	3	0000110	0110000	Yes
4	0100	4	1001100	0011001	Yes
5	0101	5	0100100	0010010	Yes
6	0110	6	0100000	0000010	Yes
7	0111	7	0001111	1111000	Yes
8	1000	8	0000000	0000000	Yes
9	1001	9	0000100	0010000	Yes
10	1010	A	0001000	0001000	No (X)
11	1011	b	1100000	0000011	No (X)
12	1100	C	0110001	1000110	No (X)
13	1101	d	1000010	0100001	No (X)
14	1110	E	0110000	0000110	No (X)
15	1111	F	0111000	0001110	No (X)

1.3 How to read and simplify every K-map

1. Rows use DC and columns use BA, both ordered $00, 01, 11, 10$. The small gray number inside each cell is its minterm index $m = 8D + 4C + 2B + A$.
2. Enter the target output values. Since the functions being minimized are the actual active-low outputs, **group the 1s, which mean segment off**. Do not group the illuminated 0s for the SOP functions shown here. X cells may assist a group but do not have to be covered.
3. Form rectangular groups of 1, 2, 4, 8, or 16 cells. Groups may overlap and wrap across opposite boundaries, but cannot contain a specified 0. Boundary pieces of the same color belong to one group.
4. Retain variables that remain fixed within a group. A fixed 1 gives the variable; a fixed 0 gives its complement. Eliminate changing variables. A group of 2^k cells eliminates k variables. For example, $\{4, 5, 12, 13\}$ fixes $C = 1, B = 0$, giving $C\overline{B}$.
5. OR the product terms from the groups to obtain a sum of products (SOP). The table under each map lists every group, its covered cells, its fixed variables, and the resulting term. This supplies the elimination steps for each output.

Each output is minimized independently: first minimize the number of terms, then the number of literals among solutions with that term count. The supplied checker independently enumerates all $3^4 = 81$ cubes and solves an exact covering problem by dynamic programming. Sharing is then applied by reusing identical terms in the selected SOPs. This is **not a claim of globally optimal multi-output logic**. Two-input gate decomposition, fan-out, and FPGA LUT mapping use different cost models.

2 Prelab 1: BCD-to-seven-segment decoder

For all seven functions, the don't-care set is $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$. The following maps show the full minimization.

BCD: output a

$a = \sum m\{1, 4\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

$DC \backslash BA$	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	X	X	X	X
10	0	0	X	X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{4, 12\}$	$C = 1, B = 0, A = 0$	$C\overline{B}\overline{A}$
G_2	$\{1\}$	$D = 0, C = 0, B = 0, A = 1$	$\overline{D}\overline{C}\overline{B}A$

OR the group terms: $\boxed{a = C\overline{B}\overline{A} + \overline{D}\overline{C}\overline{B}A}$.
Cost: 2 product term(s), 7 literal(s).

BCD: output b

$b = \sum m\{5, 6\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

$DC \backslash BA$	00	01	11	10
00	0	0	0	0
01	0	1	0	1
11	X	X	X	X
10	0	0	X	X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{5, 13\}$	$C = 1, B = 0, A = 1$	$C\overline{B}A$
G_2	$\{6, 14\}$	$C = 1, B = 1, A = 0$	$CB\overline{A}$

OR the group terms: $\boxed{b = C\overline{B}A + CB\overline{A}}$.
Cost: 2 product term(s), 6 literal(s).

BCD: output c

$c = \sum m\{2\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

$DC \backslash BA$	00	01	11	10
00	0	0	0	1
01	0	0	0	0
11	X	X	X	X
10	0	0	X	X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{2, 10\}$	$C = 0, B = 1, A = 0$	$\overline{C}B\overline{A}$

OR the group terms: $\boxed{c = \overline{C}B\overline{A}}$.
Cost: 1 product term(s), 3 literal(s).

BCD: output d

$d = \sum m\{1, 4, 7\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

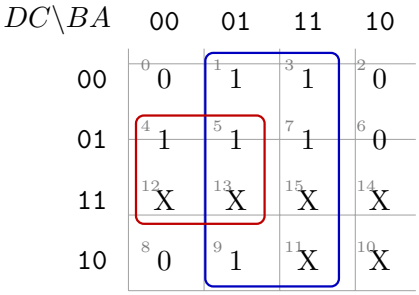
$DC \backslash BA$	00	01	11	10
00	0	1	0	0
01	1	0	1	0
11	X	X	X	X
10	0	0	X	X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{7, 15\}$	$C = 1, B = 1, A = 1$	CBA
G_2	$\{4, 12\}$	$C = 1, B = 0, A = 0$	$C\overline{B}\overline{A}$
G_3	$\{1\}$	$D = 0, C = 0, B = 0, A = 1$	$\overline{D}\overline{C}\overline{B}A$

OR the group terms: $\boxed{d = CBA + C\overline{B}\overline{A} + \overline{D}\overline{C}\overline{B}A}$.
Cost: 3 product term(s), 10 literal(s).

BCD: output e

$e = \sum m\{1, 3, 4, 5, 7, 9\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

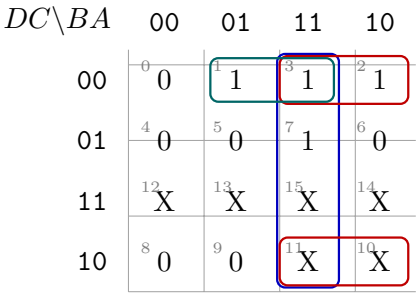


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{1, 3, 5, 7, 9, 11, 13, 15\}$	$A = 1$	A
G_2	$\{4, 5, 12, 13\}$	$C = 1, B = 0$	$C\overline{B}$

OR the group terms: $\boxed{e = A + C\overline{B}}$.
Cost: 2 product term(s), 3 literal(s).

BCD: output f

$f = \sum m\{1, 2, 3, 7\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.



Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{3, 7, 11, 15\}$	$B = 1, A = 1$	BA
G_2	$\{2, 3, 10, 11\}$	$C = 0, B = 1$	$\overline{C}B$
G_3	$\{1, 3\}$	$D = 0, C = 0, A = 1$	$\overline{DC}A$

OR the group terms: $\boxed{f = BA + \overline{C}B + \overline{DC}A}$.
Cost: 3 product term(s), 7 literal(s).

BCD: output g

$g = \sum m\{0, 1, 7\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

$DC \backslash BA$	00	01	11	10
00	0: 1	1: 1	3: 0	2: 0
01	4: 0	5: 0	7: 1	6: 0
11	12: X	13: X	15: X	14: X
10	8: 0	9: 0	11: X	10: X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{7, 15\}$	$C = 1, B = 1, A = 1$	CBA
G_2	$\{0, 1\}$	$D = 0, C = 0, B = 0$	$\overline{DCB}$

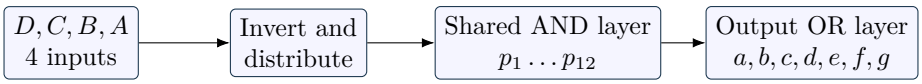
OR the group terms: $\boxed{g = CBA + \overline{DCB}}$.
Cost: 2 product term(s), 6 literal(s).

2.1 BCD decoder: complete gate connections

Generate $\overline{D}, \overline{C}, \overline{B}, \overline{A}$ with four shared inverters, then distribute both polarities of the four inputs. Each row in the left table defines one product node: use a multi-input AND for a multi-literal term, or a direct wire for a single literal. The right table gives every output connection. Use an OR gate for each sum of multiple nodes. These signals are already active-low segment controls; do not invert all seven outputs again.

Node	AND inputs / direct wire
p_1	$C\overline{B}\overline{A}$
p_2	$\overline{D}\overline{C}BA$
p_3	$C\overline{B}A$
p_4	$CB\overline{A}$
p_5	$\overline{C}B\overline{A}$
p_6	CBA
p_7	A
p_8	$C\overline{B}$
p_9	BA
p_{10}	$\overline{C}B$
p_{11}	$\overline{D}\overline{C}A$
p_{12}	$\overline{D}\overline{C}\overline{B}$

Output	OR inputs / wire
a	$p_1 + p_2$
b	$p_3 + p_4$
c	p_5
d	$p_6 + p_1 + p_2$
e	$p_7 + p_8$
f	$p_9 + p_{10} + p_{11}$
g	$p_6 + p_{12}$



The block diagram shows the layers; the two tables specify all individual nets. An FPGA maps the logic to LUTs, so discrete gate counts are not Quartus LE counts.

2.2 Sharing product terms between outputs

Define

$$P = C\overline{A}\overline{B}, \qquad Q = A\overline{B}\overline{C}\overline{D}, \qquad R = ABC.$$

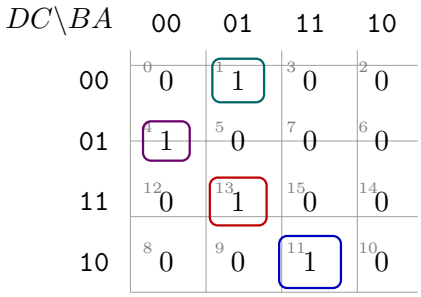
Then $a = P + Q$, $d = P + Q + R$, and $g = R + \overline{B}\overline{C}\overline{D}$. Without sharing, P and Q are each generated separately for a and d; R is generated separately for d and g. Reusing each of these products saves **three multi-input AND gates**, under the convention that one product is implemented by one AND gate. The input inverters remain shared and the output OR gates are unchanged. For a single example, sharing P between a and d saves one three-input AND gate.

3 Prelab 2: complete hexadecimal decoder

Combining digits 0–9 with A, b, C, d, E, and F defines all sixteen rows. There are no don't-cares. Added rows can require either 0 or 1 for a segment, so a BCD grouping can become invalid. For example, $C\overline{AB}$ includes input 12, but glyph C requires output a to be 0; therefore the corresponding a term needs an additional $\overline{D}$ restriction.

Hex: output a

$a = \sum m\{1, 4, 11, 13\}$; no don't-cares.

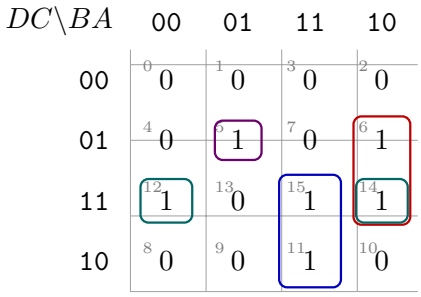


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{11\}$	$D = 1, C = 0, B = 1, A = 1$	$D\overline{C}BA$
G_2	$\{13\}$	$D = 1, C = 1, B = 0, A = 1$	$DC\overline{B}A$
G_3	$\{1\}$	$D = 0, C = 0, B = 0, A = 1$	$\overline{DCB}A$
G_4	$\{4\}$	$D = 0, C = 1, B = 0, A = 0$	$\overline{D}C\overline{BA}$

OR the group terms: $\boxed{a = D\overline{C}BA + DC\overline{B}A + \overline{DCB}A + \overline{D}C\overline{BA}}$.
Cost: 4 product term(s), 16 literal(s).

Hex: output b

$b = \sum m\{5, 6, 11, 12, 14, 15\}$; no don't-cares.

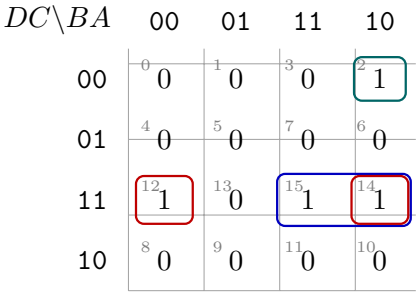


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{11, 15\}$	$D = 1, B = 1, A = 1$	DBA
G_2	$\{6, 14\}$	$C = 1, B = 1, A = 0$	$CB\overline{A}$
G_3	$\{12, 14\}$	$D = 1, C = 1, A = 0$	$DC\overline{A}$
G_4	$\{5\}$	$D = 0, C = 1, B = 0, A = 1$	$\overline{DCB}A$

OR the group terms: $\boxed{b = DBA + CB\overline{A} + DC\overline{A} + \overline{DCB}A}$.
Cost: 4 product term(s), 13 literal(s).

Hex: output c

$c = \sum m\{2, 12, 14, 15\}$; no don't-cares.

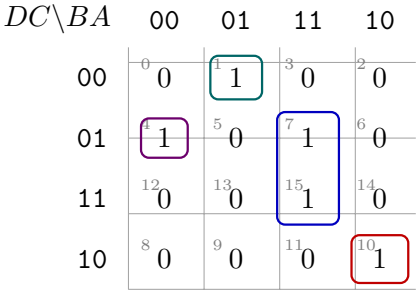


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{14, 15\}$	$D = 1, C = 1, B = 1$	DCB
G_2	$\{12, 14\}$	$D = 1, C = 1, A = 0$	$DC\overline{A}$
G_3	$\{2\}$	$D = 0, C = 0, B = 1, A = 0$	$\overline{DC}B\overline{A}$

OR the group terms: $\boxed{c = DCB + DC\overline{A} + \overline{DC}B\overline{A}}$.
Cost: 3 product term(s), 10 literal(s).

Hex: output d

$d = \sum m\{1, 4, 7, 10, 15\}$; no don't-cares.

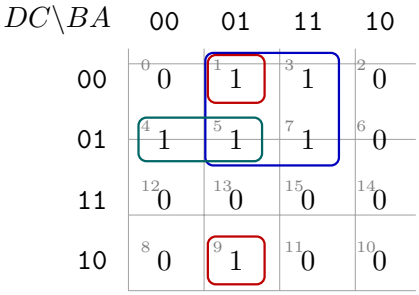


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{7, 15\}$	$C = 1, B = 1, A = 1$	CBA
G_2	$\{10\}$	$D = 1, C = 0, B = 1, A = 0$	$D\overline{C}B\overline{A}$
G_3	$\{1\}$	$D = 0, C = 0, B = 0, A = 1$	$\overline{DC}\overline{B}A$
G_4	$\{4\}$	$D = 0, C = 1, B = 0, A = 0$	$\overline{D}C\overline{BA}$

OR the group terms: $\boxed{d = CBA + D\overline{C}B\overline{A} + \overline{DC}\overline{B}A + \overline{D}C\overline{BA}}$.
Cost: 4 product term(s), 15 literal(s).

Hex: output e

$e = \sum m\{1, 3, 4, 5, 7, 9\}$; no don't-cares.

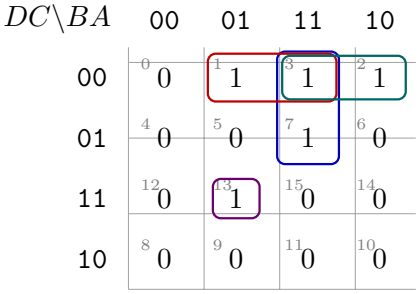


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{1, 3, 5, 7\}$	$D = 0, A = 1$	$\overline{D}A$
G_2	$\{1, 9\}$	$C = 0, B = 0, A = 1$	$\overline{C}\overline{B}A$
G_3	$\{4, 5\}$	$D = 0, C = 1, B = 0$	$\overline{D}C\overline{B}$

OR the group terms: $\boxed{e = \overline{D}A + \overline{C}\overline{B}A + \overline{D}C\overline{B}}$.
Cost: 3 product term(s), 8 literal(s).

Hex: output f

$f = \sum m\{1, 2, 3, 7, 13\}$; no don't-cares.

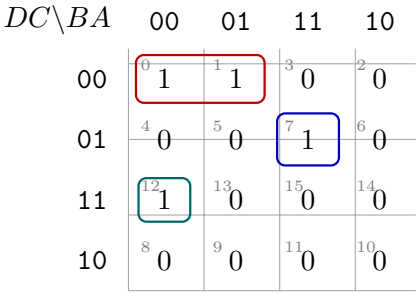


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{3, 7\}$	$D = 0, B = 1, A = 1$	$\overline{D}BA$
G_2	$\{1, 3\}$	$D = 0, C = 0, A = 1$	$\overline{D}\overline{C}A$
G_3	$\{2, 3\}$	$D = 0, C = 0, B = 1$	$\overline{D}\overline{C}B$
G_4	$\{13\}$	$D = 1, C = 1, B = 0, A = 1$	$DC\overline{B}A$

OR the group terms: $\boxed{f = \overline{D}BA + \overline{D}\overline{C}A + \overline{D}\overline{C}B + DC\overline{B}A}$.
Cost: 4 product term(s), 13 literal(s).

Hex: output g

$g = \sum m\{0, 1, 7, 12\}$; no don't-cares.



Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{7\}$	$D = 0, C = 1, B = 1, A = 1$	$\overline{D}CBA$
G_2	$\{0, 1\}$	$D = 0, C = 0, B = 0$	$\overline{DCB}$
G_3	$\{12\}$	$D = 1, C = 1, B = 0, A = 0$	$DC\overline{BA}$

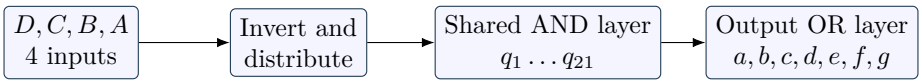
OR the group terms: $\boxed{g = \overline{D}CBA + \overline{DCB} + DC\overline{BA}}$.
Cost: 3 product term(s), 11 literal(s).

3.1 hex7seg: complete gate connections

Generate $\overline{D}, \overline{C}, \overline{B}, \overline{A}$ with four shared inverters, then distribute both polarities of the four inputs. Each row in the left table defines one product node: use a multi-input AND for a multi-literal term, or a direct wire for a single literal. The right table gives every output connection. Use an OR gate for each sum of multiple nodes. These signals are already active-low segment controls; do not invert all seven outputs again.

Node	AND inputs / direct wire
q_1	$D\overline{C}BA$
q_2	$DC\overline{B}A$
q_3	$\overline{D}C\overline{B}A$
q_4	$\overline{D}C\overline{B}\overline{A}$
q_5	DBA
q_6	$CB\overline{A}$
q_7	$DC\overline{A}$
q_8	$\overline{D}C\overline{B}A$
q_9	DCB
q_{10}	$\overline{D}CB\overline{A}$
q_{11}	CBA
q_{12}	$D\overline{C}B\overline{A}$
q_{13}	$\overline{D}A$
q_{14}	$\overline{C}BA$
q_{15}	$\overline{D}C\overline{B}$
q_{16}	$\overline{D}BA$
q_{17}	$\overline{D}CA$
q_{18}	$\overline{D}CB$
q_{19}	$\overline{D}CBA$
q_{20}	$\overline{D}CB$
q_{21}	$DC\overline{B}\overline{A}$

Output	OR inputs / wire
a	$q_1 + q_2 + q_3 + q_4$
b	$q_5 + q_6 + q_7 + q_8$
c	$q_9 + q_7 + q_{10}$
d	$q_{11} + q_{12} + q_3 + q_4$
e	$q_{13} + q_{14} + q_{15}$
f	$q_{16} + q_{17} + q_{18} + q_2$
g	$q_{19} + q_{20} + q_{21}$



The block diagram shows the layers; the two tables specify all individual nets. An FPGA maps the logic to LUTs, so discrete gate counts are not Quartus LE counts.

3.2 Cost comparison

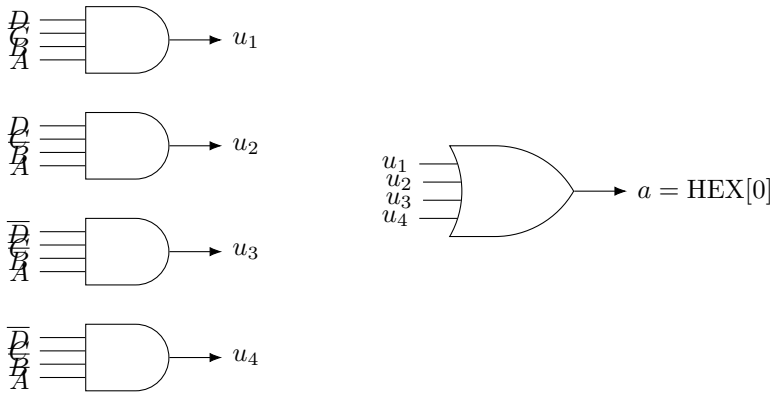
Output	BCD terms	Hex terms	BCD literals	Hex literals
a	2	4	7	16
b	2	4	6	13
c	1	3	3	10
d	3	4	10	15
e	2	3	3	8
f	3	4	7	13
g	2	3	6	11
Total	15	25	42	86

After identical-term sharing	BCD	Hex
Distinct product nodes (including direct wires)	12	21
Multi-input AND gates	11	21
Multi-input OR gates	6	7
Input inverters	4	4
Product-layer literals after sharing	32	71
Quartus Total logic elements	Pending	Pending

A literal is one occurrence of an input variable or its complement in a product; OR connections are not counted as literals. Gate counts assume arbitrary fan-in. The BCD term A is a direct wire, not an AND gate. Exact FPGA LE counts cannot be inferred from these abstract counts.

Experiment 1 also fixes $D = 0$, enabling further synthesis simplification. Consequently, its LE difference from Experiment 2 reflects both the input restriction and the decoder change. A controlled BCD-versus-Hex comparison would retain four input ports, the same FPGA target, and the same synthesis settings.

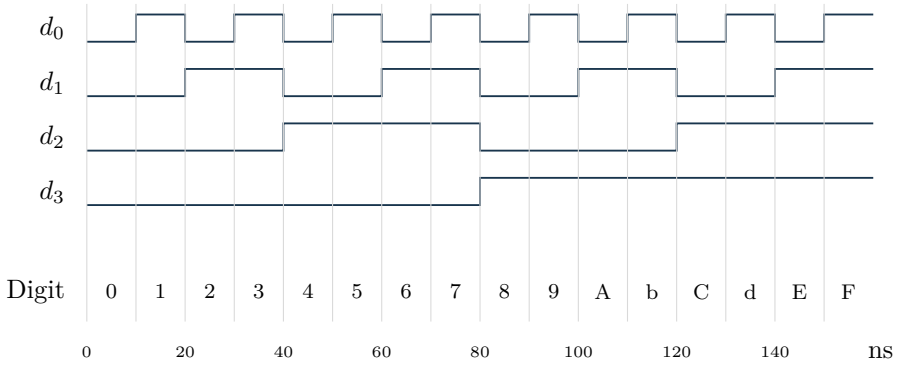
3.3 Concrete gate circuit for hexadecimal segment a



The four u_i nets correspond, in order, to the four terms in the a expression. Matching labels denote the same net. Construct the other six outputs using the full connection tables; shared product nodes feed multiple output gates.

3.4 Theoretical timing for all sixteen codes

The diagram below is a **zero-delay theoretical waveform**, not a ModelSim screenshot. Start every input at 0 at $t = 0$. Use periods of 20, 40, 80, and 160 ns for d_0, d_1, d_2, d_3, each with a 50% duty cycle. A new input code appears every 10 ns, covering all sixteen codes during $[0, 160)$ ns.



Interval / ns	$DCBA$	Expected glyph	Expected $abcdefg$
$[0, 10)$	0000	0	0000001
$[10, 20)$	0001	1	1001111
$[20, 30)$	0010	2	0010010
$[30, 40)$	0011	3	0000110
$[40, 50)$	0100	4	1001100
$[50, 60)$	0101	5	0100100
$[60, 70)$	0110	6	0100000
$[70, 80)$	0111	7	0001111
$[80, 90)$	1000	8	0000000
$[90, 100)$	1001	9	0000100
$[100, 110)$	1010	A	0001000
$[110, 120)$	1011	b	1100000
$[120, 130)$	1100	C	0110001
$[130, 140)$	1101	d	1000010
$[140, 150)$	1110	E	0110000
$[150, 160)$	1111	F	0111000

In a real simulation, sample after the combinational circuit settles, for example at $5, 15, \ldots, 155$ ns. Different initial clock levels or phases change the order of the codes; always interpret the input bus rather than assuming an order.

4 Prelab 3: result and operation-letter displays

4.1 Letter patterns and select logic

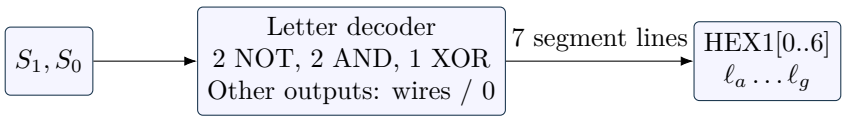
Use the operation codes from Lab 8. Lowercase o uses segments c, d, e, and g, distinguishing it from digit 0.

S_1S_0	Operation	Letter	Lit segments	$abcdefg$
00	AND	A	a,b,c,e,f,g	`0001000`
01	OR	o	c,d,e,g	`1100010`
10	XOR	E	a,d,e,f,g	`0110000`
11	ADD	d	b,c,d,e,g	`1000010`

Let the HEX1 segment outputs be $\ell_a, \ldots, \ell_g$. Reading each column of the four-row truth table gives

$$\ell_a = S_0, \qquad \ell_b = S_1 \oplus S_0, \qquad \ell_c = S_1\overline{S_0},$$
$$\ell_d = \overline{S_1}\,\overline{S_0}, \qquad \ell_e = 0, \qquad \ell_f = S_0, \qquad \ell_g = 0.$$

For example, $\ell_c = 1$ only for $S_1S_0 = 10$ (E); c is lit for the other letters. Thus $\ell_c = S_1\overline{S_0}$. Outputs ℓ_a and ℓ_f share the S_0 wire.



Alternatively, use seven 4:1 multiplexers, one per segment. All share S_1, S_0. Connect each column's four constants to I_0, I_1, I_2, I_3 in that order. For a, the constants are $(0, 1, 0, 1)$; for b, $(0, 1, 1, 0)$; e and g are always 0.

4.2 Alternative retaining a second hex7seg

If two `hex7seg` instances are required at the top level, let the second initially decode A, 0, E, and d. For $S_1S_0 = 00, 01, 10, 11$, supply $1010, 0000, 1110, 1101$, respectively. The encoding logic is

$$D_L = S_1 + \overline{S_0}, \quad C_L = S_1, \quad B_L = \overline{S_0}, \quad A_L = S_1S_0.$$

Let $q = \overline{S_1}S_0$, which is 1 only for OR, and let the second decoder outputs be $h_a, \ldots, h_g$. Override the outputs as follows:

$$\ell_a = h_a + q, \quad \ell_b = h_b + q, \quad \ell_f = h_f + q, \quad \ell_g = h_g\overline{q}, \qquad \ell_c = h_c,\ \ell_d = h_d,\ \ell_e = h_e.$$

For OR, this turns a, b, and f off and g on, changing 0 to o. The other three glyphs pass unchanged. **Only the final override outputs drive HEX1; never connect two output drivers together.** This alternative costs extra logic; the primary design uses the direct letter decoder.

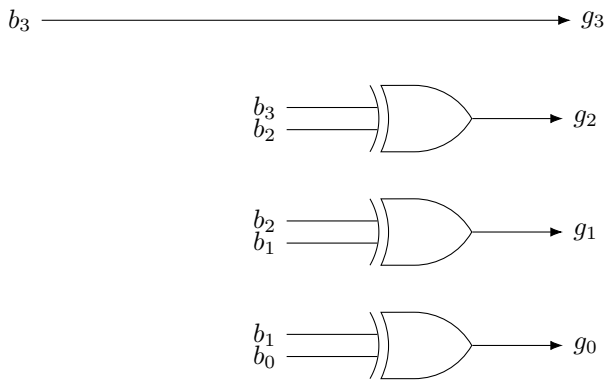
5 Prelab 4: binary-to-Gray and BCD-to-excess-3

5.1 Deriving the four Gray-code equations

Let the binary input be $b_3b_2b_1b_0$. The most significant Gray bit equals the binary MSB. Each remaining Gray bit indicates whether two neighboring binary bits differ:

$$\boxed{g_3 = b_3, \quad g_2 = b_3 \oplus b_2, \quad g_1 = b_2 \oplus b_1, \quad g_0 = b_1 \oplus b_0.}$$

The XOR truth table is $00 \mapsto 0$, $01 \mapsto 1$, $10 \mapsto 1$, $11 \mapsto 0$. Equal neighboring bits give 0; unequal bits give 1. In vector form, $G = B \oplus (B \gg 1)$. The circuit needs **three XOR gates and one direct wire**; no K-map is needed.



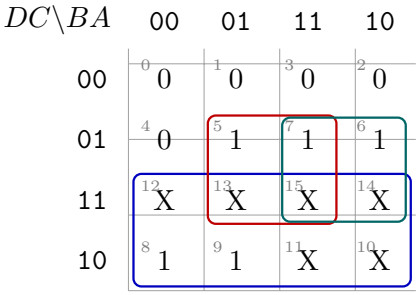
5.2 Excess-3 truth table and bit-by-bit minimization

Valid inputs are BCD digits 0–9. The target is $Z = \text{value}(DCBA) + 3$; inputs 10–15 are don't-cares. Write the output as $z_3z_2z_1z_0$.

Decimal	$DCBA$	$z_3z_2z_1z_0$	Hex glyph	Arithmetic
0	0000	0011	3	$0 + 3 = 3$
1	0001	0100	4	$1 + 3 = 4$
2	0010	0101	5	$2 + 3 = 5$
3	0011	0110	6	$3 + 3 = 6$
4	0100	0111	7	$4 + 3 = 7$
5	0101	1000	8	$5 + 3 = 8$
6	0110	1001	9	$6 + 3 = 9$
7	0111	1010	A	$7 + 3 = 10$
8	1000	1011	b	$8 + 3 = 11$
9	1001	1100	C	$9 + 3 = 12$

Excess-3: output z_3

$z_3 = \sum m\{5, 6, 7, 8, 9\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

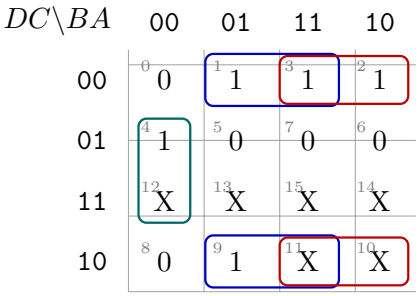


Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{8, 9, 10, 11, 12, 13, 14, 15\}$	$D = 1$	D
G_2	$\{5, 7, 13, 15\}$	$C = 1, A = 1$	CA
G_3	$\{6, 7, 14, 15\}$	$C = 1, B = 1$	CB

OR the group terms: $\boxed{z_3 = D + CA + CB}$.
Cost: 3 product term(s), 5 literal(s).

Excess-3: output z_2

$z_2 = \sum m\{1, 2, 3, 4, 9\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.



Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{1, 3, 9, 11\}$	$C = 0, A = 1$	$\overline{C}A$
G_2	$\{2, 3, 10, 11\}$	$C = 0, B = 1$	$\overline{C}B$
G_3	$\{4, 12\}$	$C = 1, B = 0, A = 0$	$C\overline{BA}$

OR the group terms: $\boxed{z_2 = \overline{C}A + \overline{C}B + C\overline{BA}}$.
Cost: 3 product term(s), 7 literal(s).

Excess-3: output z_1

$z_1 = \sum m\{0, 3, 4, 7, 8\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

$DC \backslash BA$	00	01	11	10
00	0: 1	1: 0	3: 1	2: 0
01	4: 1	5: 0	7: 1	6: 0
11	12: X	13: X	15: X	14: X
10	8: 1	9: 0	11: X	10: X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{3, 7, 11, 15\}$	$B = 1, A = 1$	BA
G_2	$\{0, 4, 8, 12\}$	$B = 0, A = 0$	$\overline{BA}$

OR the group terms: $\boxed{z_1 = BA + \overline{BA}}$.
Cost: 2 product term(s), 4 literal(s).

Excess-3: output z_0

$z_0 = \sum m\{0, 2, 4, 6, 8\}$; $\mathcal{D} = \{10, 11, 12, 13, 14, 15\}$.

$DC \backslash BA$	00	01	11	10
00	0: 1	1: 0	3: 0	2: 1
01	4: 1	5: 0	7: 0	6: 1
11	12: X	13: X	15: X	14: X
10	8: 1	9: 0	11: X	10: X

Group	Cells covered (incl. X)	Fixed variables	Product term
G_1	$\{0, 2, 4, 6, 8, 10, 12, 14\}$	$A = 0$	$\overline{A}$

OR the group terms: $\boxed{z_0 = \overline{A}}$.
Cost: 1 product term(s), 1 literal(s).

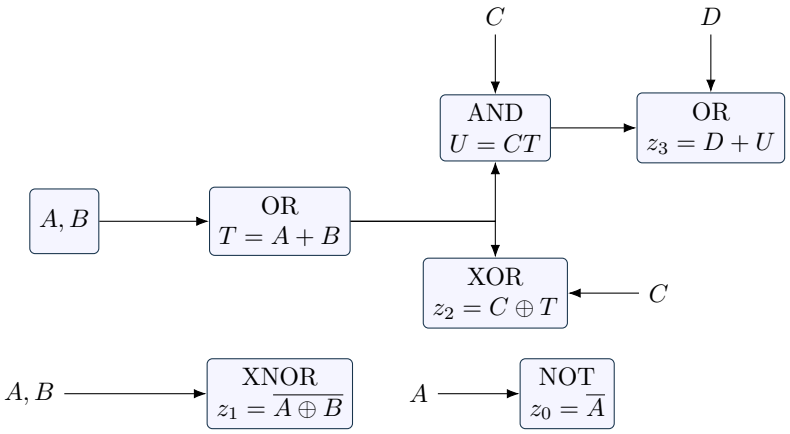
5.3 Complete excess-3 connections

Combining the four maps gives

$$z_3 = D + AC + BC = D + C(A + B),$$
$$z_2 = A\overline{C} + B\overline{C} + C\overline{AB} = \overline{C}(A + B) + C\overline{(A + B)} = C \oplus (A + B),$$
$$z_1 = AB + \overline{AB} = \overline{A \oplus B},$$
$$z_0 = \overline{A}.$$

Introduce $T = A + B$ and $U = CT$ to reuse intermediate logic. These equations are specified for valid BCD inputs only: **inputs 10–15 are not required to produce input plus 3**.

Node	Gate / connection	Destination
T	OR(A, B)	AND input for U; XOR input for z_2
U	AND(C, T)	OR input for z_3
z_3	OR(D, U)	Output MSB
z_2	XOR(C, T)	Output bit 2
z_1	XNOR(A, B)	Output bit 1
z_0	NOT(A)	Output LSB



If an XNOR gate is unavailable, use XOR followed by NOT. This factored network is multilevel, so its gate cost differs from the independent two-level SOP term count.

5.4 Prelab question: common anode or common cathode?

Table 2 represents a **common-anode display with active-low controls**, as specified by the handout. For example, digit 0 lights a–f, whose entries are all 0, while g is 1 and stays off. A common-cathode interface normally uses active-high segment signals; invert the seven outputs if the target interface requires the opposite polarity.

6 Postlab: operating principles and experiment connections

6.1 Operating principles, time, and handout issues

All circuits are combinational. A decoder maps a numeric input to seven segment-control bits; the ALU produces a numeric result for the selected operation; Gray and excess-3 converters change the representation. After propagation settles, the outputs depend only on the current inputs. No board clock or register is required. The clocks in the timing example are simulation stimuli for input enumeration.

Actual total time spent: _____ hours. TA demonstration / sign-off: _____.

Handout issues include the prose/Table 2 conflict for digits 6 and 9, and the unclear relationship between two `hex7seg` symbols and the separate letter decoder in Experiment 3. In Experiment 4, "a third hex7seg" should not be read as requiring two unused instances in that independent project: only two are needed, for HEX2 and HEX3.

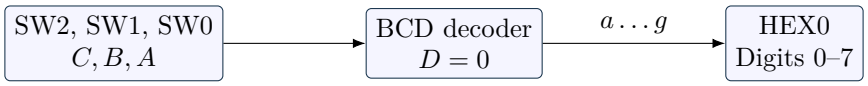
6.2 Global connection rules

- This report specifies **logical board ports**: SW, KEY, LEDR, and HEX. Physical FPGA `PIN_xxx` assignments must come from the course's matching DE0-CV pin table. Those physical pin numbers have not been checked or invented here.
- To blank an unused seven-segment display, set all seven outputs to 1. Unused LEDs may be set to 0. Use explicit logic constants for 0 and 1; do not leave inputs floating.
- This design inverts each active-low KEY input: $a_1 = \overline{KEY3}$, $a_0 = \overline{KEY2}$, $b_1 = \overline{KEY1}$, $b_0 = \overline{KEY0}$. Thus a pressed button represents operand bit 1 and a released button represents 0.
- Connect by segment index: a to [0], through g to [6]. Do not confuse an $abcdefg$ pattern with the order of the bus $[6:0]$.

6.3 Experiment 1: displaying a three-bit number

Online circuit illustrations and clickable sources appear in Appendix C, with differences from this experiment noted.

Use the BCD decoder with $D = 0$, $C = SW2$, $B = SW1$, and $A = SW0$. Connect outputs a–g to HEX0[0]–HEX0[6], respectively. Generate each complemented input with NOT gates.



Set the switches to each value from 000 through 111. Compare the settled display with the expected digit, and record the actual observation separately.

SW2 SW1 SW0	Expected digit	$abcdefg$	Actual HEX0	Match?
000	0	0000001	Pending	Pending
001	1	1001111	Pending	Pending
010	2	0010010	Pending	Pending
011	3	0000110	Pending	Pending
100	4	1001100	Pending	Pending
101	5	0100100	Pending	Pending
110	6	0100000	Pending	Pending
111	7	0001111	Pending	Pending

Photo to add: HEX0 showing a digit, with the corresponding switch positions visible.

6.4 Experiment 2: complete hexadecimal display

Online circuit illustrations and clickable sources appear in Appendix C, with differences from this experiment noted.

Replace the BCD decoder with `hex7seg`. Keep the lower three switch connections and connect D to SW3. Output a still goes to HEX0[0], through g to HEX0[6]. Verify all sixteen rows in the target table.

SW3–SW0	Expected glyph	$abcdefg$	Actual HEX0	Match?
0000	0	0000001	Pending	Pending
0001	1	1001111	Pending	Pending
0010	2	0010010	Pending	Pending
0011	3	0000110	Pending	Pending
0100	4	1001100	Pending	Pending
0101	5	0100100	Pending	Pending
0110	6	0100000	Pending	Pending
0111	7	0001111	Pending	Pending
1000	8	0000000	Pending	Pending
1001	9	0000100	Pending	Pending
1010	A	0001000	Pending	Pending
1011	b	1100000	Pending	Pending
1100	C	0110001	Pending	Pending
1101	d	1000010	Pending	Pending
1110	E	0110000	Pending	Pending
1111	F	0111000	Pending	Pending

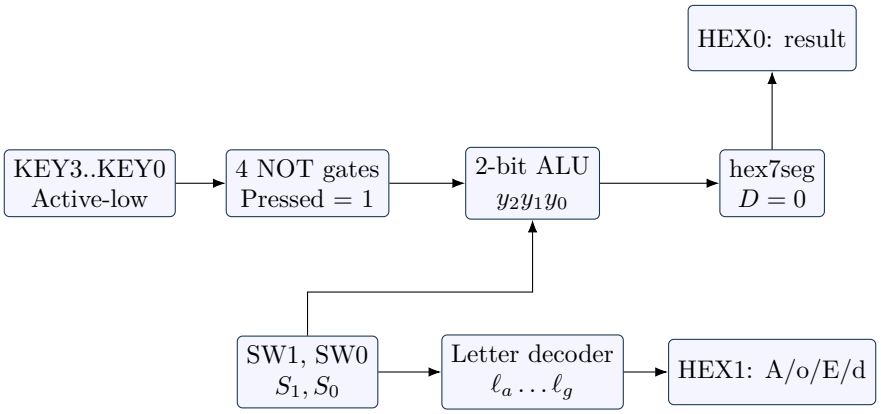
Photo to add: HEX0 displaying one of A, b, C, d, E, or F.

Experiment 1 LE count: _____. Experiment 2 LE count: _____. Adding hexadecimal constraints reduces minimization freedom; the selected independent SOPs increase from 15 to 25 terms. Whether the mapped FPGA LE count increases, and by how much, requires actual compilation reports from the same tool environment.

6.5 Experiment 3: two-bit ALU and two displays

Online circuit illustrations and clickable sources appear in Appendix C, with differences from this experiment noted.

Input and output connections. Invert the four KEY signals as described above. Connect $S_1 = SW1$, $S_0 = SW0$. Connect ALU outputs y_2, y_1, y_0 to decoder inputs C, B, A, tie $D = 0$, and send its segment outputs to HEX0. Send S_1, S_0 also to the letter decoder and connect its seven outputs to HEX1.



Complete ALU logic. The following equations allow reconstruction without an existing Quartus project. First form the two-bit arithmetic sum:

$$t_0 = a_0 \oplus b_0, \qquad c_1 = a_0 b_0,$$
$$t_1 = a_1 \oplus b_1 \oplus c_1, \qquad c_2 = a_1 b_1 + a_1 c_1 + b_1 c_1.$$

For $i = 0, 1$, use one 4:1 MUX with $I_0 = a_i b_i$, $I_1 = a_i + b_i$, $I_2 = a_i \oplus b_i$, and $I_3 = t_i$. Both MUXs share S_1, S_0; their outputs are y_i. The highest result bit is $y_2 = S_1 S_0 c_2$, forcing it to 0 for logic operations. This implementation is functionally equivalent to the two-slice ALU in Lab 8.

Button example. For arithmetic $3 + 3 = 6$, set SW1=SW0=1 and press all four buttons. After inversion, $a = b = 11$ and $y = 110$. HEX0 should display 6 and HEX1 should display d. Since HEX1 is to the left of HEX0, the board reads "d6" from left to right.

Complete theoretical results: all 64 ALU cases

Each result is $y_2y_1y_0$. The displayed digit is the unsigned value of those three bits. All entries below are theoretical expectations; the last column is for measurement.

a_1a_0	b_1b_0	AND / A	OR / o	XOR / E	ADD / d	Measured?
00	00	000	000	000	000	Pending
00	01	000	001	001	001	Pending
00	10	000	010	010	010	Pending
00	11	000	011	011	011	Pending
01	00	000	001	001	001	Pending
01	01	001	001	000	010	Pending
01	10	000	011	011	011	Pending
01	11	001	011	010	100	Pending
10	00	000	010	010	010	Pending
10	01	000	011	011	011	Pending
10	10	010	010	000	100	Pending
10	11	010	011	001	101	Pending
11	00	000	011	011	011	Pending
11	01	001	011	010	100	Pending
11	10	010	011	001	101	Pending
11	11	011	011	000	110	Pending

Sixteen operand pairs times four modes give 64 cases. ADD ranges from 0 to 6; logic results range from 0 to 3. Include carry cases such as $1 + 3$, $2 + 2$, $2 + 3$, and $3 + 3$, and confirm $y_2 = 0$ in each logic mode.

Photo to add: an ADD result greater than 3 with d beside it, for example HEX1=d and HEX0=6.

Description of the letter decoder

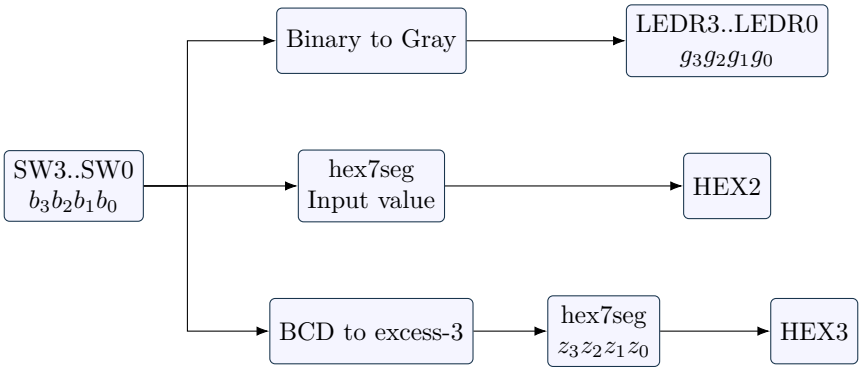
The design assigns active-low patterns to $00 \rightarrow A$, $01 \rightarrow o$, $10 \rightarrow E$, $11 \rightarrow d$, then simplifies each segment function. It uses one XOR, two two-input ANDs, and two NOTs; a/f share the S_0 wire and e/g share constant 0. The same select inputs drive the ALU and the letters, so the operation label tracks the selected function. If the second-hex7seg override circuit is used, describe that actual implementation instead.

6.6 Experiment 4: two converters sharing one input

Online circuit illustrations and clickable sources appear in Appendix C, with differences from this experiment noted.

Connections:

1. Set $b_3 = SW3$, $b_2 = SW2$, $b_1 = SW1$, $b_0 = SW0$. Fan out these signals to the Gray converter, the excess-3 converter, and the input-display decoder.
2. Connect g_3, g_2, g_1, g_0 to LEDR3, LEDR2, LEDR1, LEDR0, respectively.
3. Connect b_3, b_2, b_1, b_0 to D, C, B, A of one hex7seg; connect its outputs to HEX2.
4. Connect z_3, z_2, z_1, z_0 to D, C, B, A of another hex7seg; connect its outputs to HEX3. Do not tie z_3 to 0: unlike Experiment 1, the full four-bit value is required.



Gray-code results and the changed bit

The changed bit compares two consecutive **settled** Gray outputs. XOR the current code with the previous code to locate that bit. The first row has no predecessor.

$b_3b_2b_1b_0$	Expected $g_3g_2g_1g_0$	Changed bit	Actual LEDs	Match?
0000	0000	None (first)	Pending	Pending
0001	0001	g_0	Pending	Pending
0010	0011	g_1	Pending	Pending
0011	0010	g_0	Pending	Pending
0100	0110	g_2	Pending	Pending
0101	0111	g_0	Pending	Pending
0110	0101	g_1	Pending	Pending
0111	0100	g_0	Pending	Pending
1000	1100	g_3	Pending	Pending
1001	1101	g_0	Pending	Pending
1010	1111	g_1	Pending	Pending
1011	1110	g_0	Pending	Pending
1100	1010	g_2	Pending	Pending
1101	1011	g_0	Pending	Pending
1110	1001	g_1	Pending	Pending
1111	1000	g_0	Pending	Pending

Transient behavior. Adjacent target Gray codes differ in one bit, but this combinational converter is not guaranteed glitch-free during mechanical switch changes. Changing binary 0111 to 1000 requires four switches and can pass through other intermediate inputs. Compare outputs only after the intended input has settled. Synchronous sampling, debouncing, or a direct Gray-sequence generator would be needed for stronger timing requirements.

Excess-3 display results

Input	BCD	Excess-3	Expected HEX2	Expected HEX3	Measured?
0	0000	0011	0	3	Pending
1	0001	0100	1	4	Pending
2	0010	0101	2	5	Pending
3	0011	0110	3	6	Pending
4	0100	0111	4	7	Pending
5	0101	1000	5	8	Pending
6	0110	1001	6	9	Pending
7	0111	1010	7	A	Pending
8	1000	1011	8	b	Pending
9	1001	1100	9	C	Pending

For input 5, HEX2 shows 5 and HEX3 shows 8. For inputs 7, 8, and 9, HEX3 shows A, b, and C. It displays the four-bit excess-3 code as a hexadecimal glyph; it does not expand values 10–12 into two decimal digits. Gray remains defined for inputs 10–15, but those inputs are invalid BCD and are outside excess-3 acceptance.

Photo to add: HEX2 showing input 5 and HEX3 showing the corresponding excess-3 value 8.

7 Postlab: answers to the four questions

7.1 Why does the two-bit sum fit one digit? What about a three-bit ALU?

Unsigned two-bit operands range from 0 to 3, so the maximum sum is $3 + 3 = 6 = 110_2$. Three binary result bits are needed, but the numeric value is less than 10, so one decimal seven-segment digit is sufficient.

Unsigned three-bit operands range from 0 to 7, giving a maximum sum of $7 + 7 = 14 = 1110_2$. The result needs four binary bits. For **decimal display**, use two digits and binary-to-decimal/BCD conversion: for $N \in [0, 14]$, set the tens digit to $t = [N \geq 10]$ and the units digit to $u = N - 10t$, then decode t and u separately. For **hexadecimal display**, one existing hex7seg is still sufficient; the maximum value 14 appears as E. Therefore, two display digits are required for decimal output, not for every possible output base.

7.2 Which product term was shared, and how many gates were saved?

The BCD functions include

$$a = C\overline{AB} + A\overline{BCD}, \qquad d = C\overline{AB} + A\overline{BCD} + ABC.$$

Sharing the first two terms saves two multi-input AND gates. Sharing ABC between d and g saves one more, for **three saved AND gates** under the stated cost model.

For one specific example, $C\overline{AB}$ is reused by a and d, reducing two three-input ANDs to one. If only two-input ANDs are allowed, a three-literal product normally needs two cascaded ANDs, so sharing that product would save two two-input ANDs instead. State the counting convention. Fan-out buffers are excluded, and these numbers do not directly determine FPGA LE savings.

7.3 Why can six extra hexadecimal rows increase decoder cost?

BCD inputs 10–15 are don't-cares and can be assigned whichever values aid simplification. Hexadecimal decoding fixes them to letter patterns. Some large groups extending into the don't-care region become invalid and must be replaced by smaller groups, increasing terms or literals. The number of **input constraints** increases even though the number of outputs stays seven.

Here the selected independently minimized SOPs rise from 15 to 25 total terms. Individual outputs, alternative glyph choices, or mapped FPGA LE counts need not increase strictly. For a fixed exact cost model, adding constraints cannot enlarge the feasible implementation set; the BCD function could already use a full Hex implementation, but its don't-cares allow a cheaper choice.

7.4 Why are K-map rows and columns labeled in Gray order?

The order $00, 01, 11, 10$ makes each adjacent pair differ in exactly one variable, including the wraparound pair. Thus adjacent minterms can combine using $XY + X\overline{Y} = X$, eliminating one variable. Four-cell and eight-cell groups eliminate two and three variables, respectively.

In ordinary binary order $00, 01, 10, 11$, adjacent labels 01 and 10 differ in two variables. Treating them as a valid adjacent pair could include unwanted zero minterms, so that ordering does not support the standard rectangular grouping rule.

8 Verification, conclusions, and remaining evidence

8.1 Offline checks actually performed

The supplied Python generator uses the shared logic engine to generate SOPs with SymPy, then independently enumerates all cubes and solves an exact covering problem to check minimum term counts and their minimum literal counts. It evaluates every required input and compares the ALU gate equations with integer operations.

Check	Passed cases
Valid BCD inputs (all seven segments per input)	10
All Hex inputs (all seven segments per input)	16
Valid excess-3 inputs (all four bits per input)	10
Gray inputs / consecutive settled pairs	16 / 15
ALU cases (result and resulting segment pattern)	64
Letter-select modes, including the override alternative	4
Independent SOP cost checks	18

The actual results are saved in `verification.json`. PASS applies to **offline Boolean functionality and the specified SOP cost criterion**. It does not establish FPGA timing, physical pin correctness, LE counts, button debouncing, or observed display behavior. The theoretical waveform was not run in ModelSim.

8.2 Design-stage conclusions

The design derives both BCD and full hexadecimal active-low seven-segment logic. Don't-cares enable larger K-map groups, while shared product terms reduce duplicated logic. The letter decoder maps the operation selection to A/o/E/d, and the result decoder preserves the three-bit ALU result, including the ADD carry. Three adjacent-bit XORs implement binary-to-Gray conversion; the excess-3 circuit reuses the intermediate term $A + B$ through factoring.

The main design issues are output polarity, bus bit order, active-low buttons, the valid BCD domain, decimal versus hexadecimal display, and the distinction between Gray-code steady-state properties and switching transients. The equations agree with offline checks. Hardware agreement still requires the listed experiments. This is a **design-stage conclusion**, not a claim that the board experiments have been completed.

8.3 Complete after actual experimentation

- Actual problems encountered: _____
- How they were resolved: _____
- Measured differences from theory and causes: _____
- Total time and TA demonstration record: _____

8.4 Submission requirements and current status

Required material	Current status / remaining work
BCD and Hex K-maps	Complete: 14 maps, group elimination, expressions, and shared products.
Gray and excess-3 derivations	Complete, including four additional excess-3 K-maps.
Schematics and connections	Complete: TikZ diagrams, gate connection tables, and logical board-port mapping.
Simulation waveform	Theoretical timing and patterns supplied. A genuine ModelSim screenshot still requires execution.
Quartus screenshots and projects	Not created or connected, as requested. These drawings are not Quartus screenshots.
Board measurements, photos, and LE counts	Pending actual experiments; all measurement columns are explicitly marked.
Postlab questions and conclusions	All four answers and design conclusions are complete. Add actual laboratory experience later.
Three project archives and links	Provide actual decoder, ALU-display, and converter projects when available. This document package is not a substitute for those projects.

A Compilation and reproduction without Quartus

The file `lab9_prelab_postlab.tex` uses TikZ for the design diagrams and a companion images folder for the external reference illustrations. Keep that folder beside the .tex file when compiling. Compile it with XeLaTeX three times to update the contents and cross-references:

```
xelatex -interaction=nonstopmode -halt-on-error lab9_prelab_postlab.tex
xelatex -interaction=nonstopmode -halt-on-error lab9_prelab_postlab.tex
xelatex -interaction=nonstopmode -halt-on-error lab9_prelab_postlab.tex
```

The English version uses the standard article class and Latin Modern fonts; no Chinese fonts or ctex package are required. On Overleaf, select XeLaTeX and upload the .tex file together with its images folder. Common math packages, TikZ, booktabs, fancyhdr, enumitem, and hyperref are required.

Alternatively, run `build_pdf.ps1` from the package root in PowerShell. To regenerate the derivations, checks, CSV, and theoretical VCD, use Python 3.10 or later:

```
python -m pip install sympy
python build_lab9.py
```

Keep `lab9_logic.py` beside the English generator: it supplies the shared logic engine and is imported without generating the Chinese report. Regenerating overwrites the English .tex file, so back up hand edits first. The VCD contains zero-delay theoretical values; the CSV contains expected answers, not board observations.

B Source files and page references

1. `CPE_100_Lab_9.pdf`: page 2, Table 2 (BCD patterns); page 3, Table 3 (hexadecimal extension); page 4 (prelab, Gray, and excess-3); pages 5–6 (experiments and connections); page 7 (postlab, grading, and attachments).
2. Adjacent `lab8/CPE_100_Lab_8.pdf`: page 2, Table 2 (operation codes 00/01/10/11); page 3 (the two-bit ALU and its carry/result outputs).

C Online circuit illustrations and source links

This appendix collects the public circuit references discussed alongside the report. URLs below each illustration are clickable. Author attribution and scope differences are retained. These external examples do not establish compilation or board testing of the designs in this report.

C.1 Experiments 1 and 2: BCD and full hexadecimal decoders

The BCD reference supplies a native Quartus BDF file, which requires Quartus to view. The report's own derivations and connections remain the design for Experiment 1. The illustration below is Nathan Abel Ortiz's full hexadecimal gate-level decoder, useful for Experiment 2. It supports 0–F and is not the same minimization as a BCD-only 0–9 decoder. Check its glyph conventions against this report's target table.

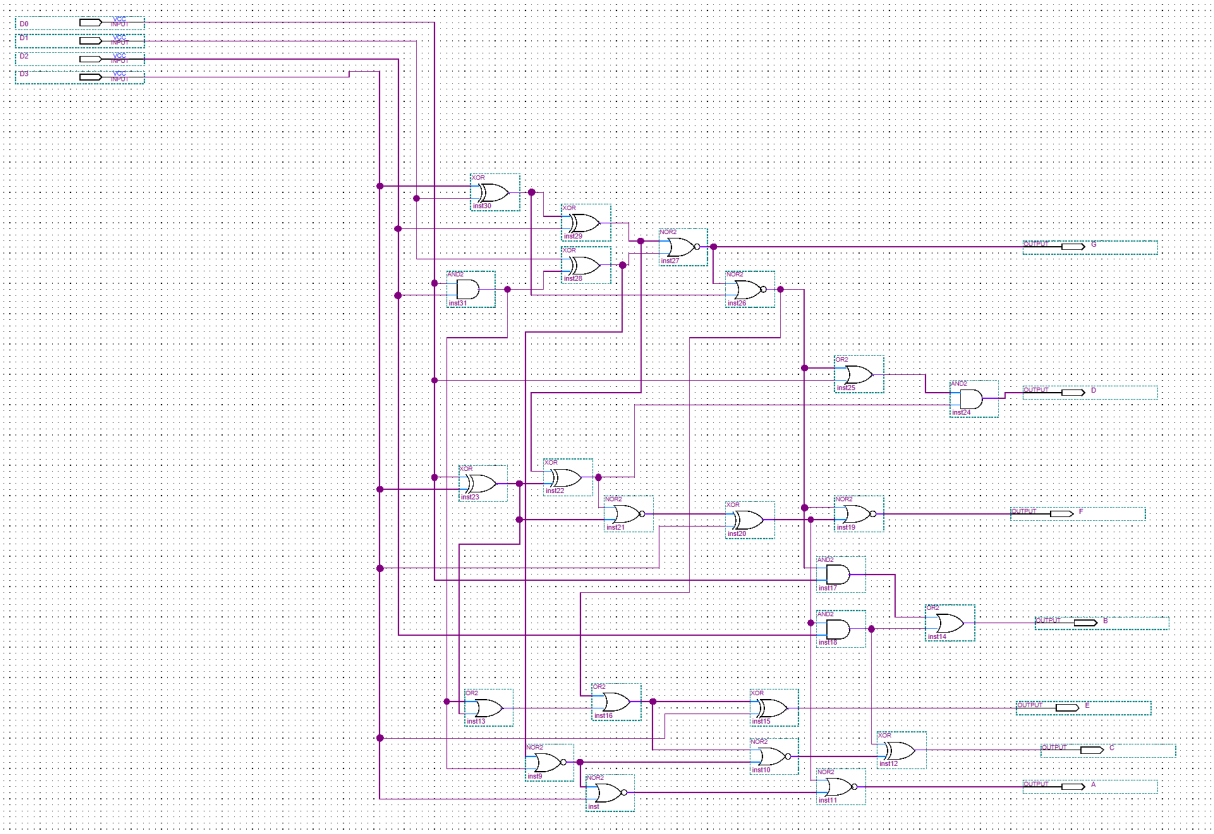


Figure 1: Nathan Abel Ortiz's Quartus hexadecimal seven-segment decoder (external reference).

Experiment 1: native BCD BDF:

https://github.com/jrmmendes/block-diagram-digital-clock/blob/master/modules/decoders/bcd_to_7segd.bdf

Experiment 2: author project page:

https://nathanabelortiz.github.io/projects/fpga-hex7.html

Original image:

https://nathanabelortiz.github.io/assets/projects/fpga/hex7-schematic.png

C.2 Experiment 3: ALU organization and display connections

The illustration below comes from the University of Jena's Experimental Hardware Projects course. It shows the operation branches and result selection of a parameterized ALU. The original diagram illustrates 64 bits; the board exercise uses four-bit inputs and ADD/SUB/AND/OR. **Our experiment uses two-bit inputs and 00 AND, 01 OR, 10 XOR, 11 ADD.** Adapt both the width and control mapping.

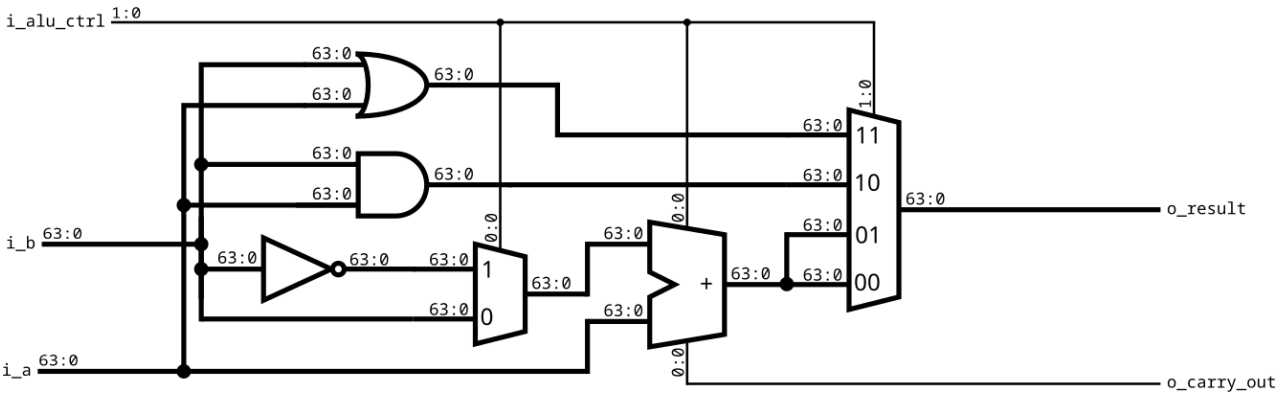


Figure 2: ALU structure from the University of Jena course (original SVG converted to PNG).

Course page: circuit and display integration:

https://scalable.uni-jena.de/opt/ehp/chapters/assignment_alu.html

Original SVG:

https://scalable.uni-jena.de/opt/ehp/_images/alu_basic.svg

Separate four-bit Quartus ALU.bdf example:

https://github.com/shashwatKanth04/Altera-Quartus-2-Basic-Designs/blob/main/ALU.bdf

The linked BDF contains AND, OR, XOR, arithmetic, and selection blocks and can serve as a hierarchy reference. Its width and functions still need adaptation. A public top-level diagram exactly matching our result on HEX0 and A/o/E/d on HEX1 was not found. Use this report's Experiment 3 dual-display diagram and letter-decoder logic for those connections.

C.3 Experiment 4: correct four-bit binary-to-Gray circuit

This diagram is from Thiago Lima's Embarcados tutorial. The most significant bit passes through; three XOR gates compute the remaining outputs for all 16 inputs. The article implements the circuit in Verilog. The same gates can be placed individually in a Quartus BDF schematic.

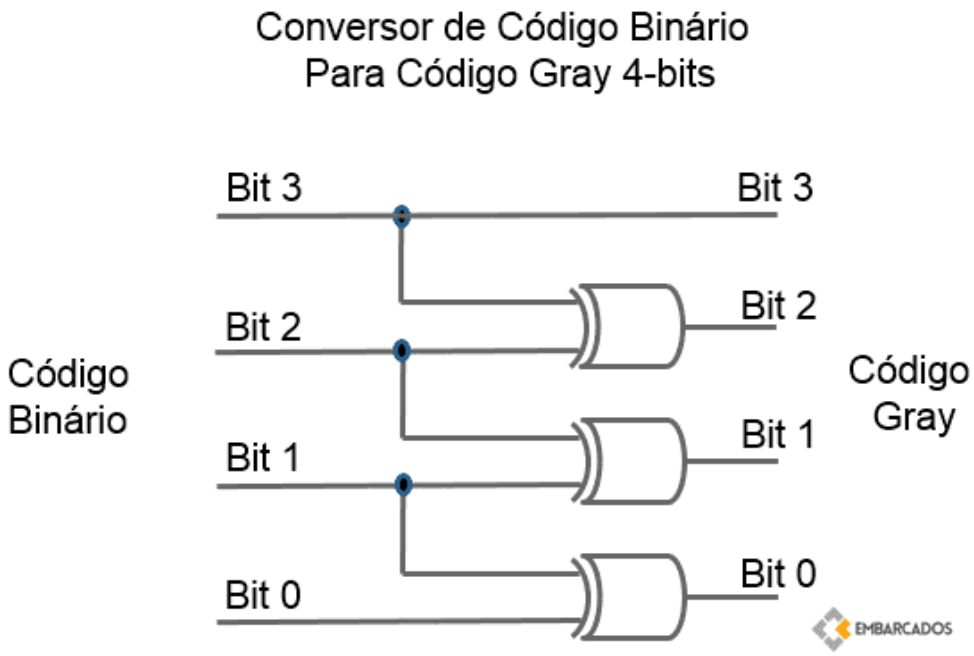


Figure 3: Four-bit binary-to-Gray converter: three adjacent-bit XORs and one direct wire. Source: Thiago Lima / Embarcados.

Tutorial and code:

https://embarcados.com.br/tutorial-de-verilog-codigo-binario-para-codigo-gray/

Original image:

https://embarcados.com.br/wp-content/uploads/2015/11/gray-7.png

For this experiment, connect SW3–SW0 to the inputs and LEDR3–LEDR0 to the outputs:

$$g_3 = b_3, \quad g_2 = b_3 \oplus b_2, \quad g_1 = b_2 \oplus b_1, \quad g_0 = b_1 \oplus b_0.$$

C.4 Experiment 4: BCD-to-excess-3 gate reference

The hand-drawn diagram below is from Mmamoonn/code-conversion-system. Its lower half implements excess-3; the upper half shows Gray conversion. This reference drawing is **not a Quartus screenshot**.

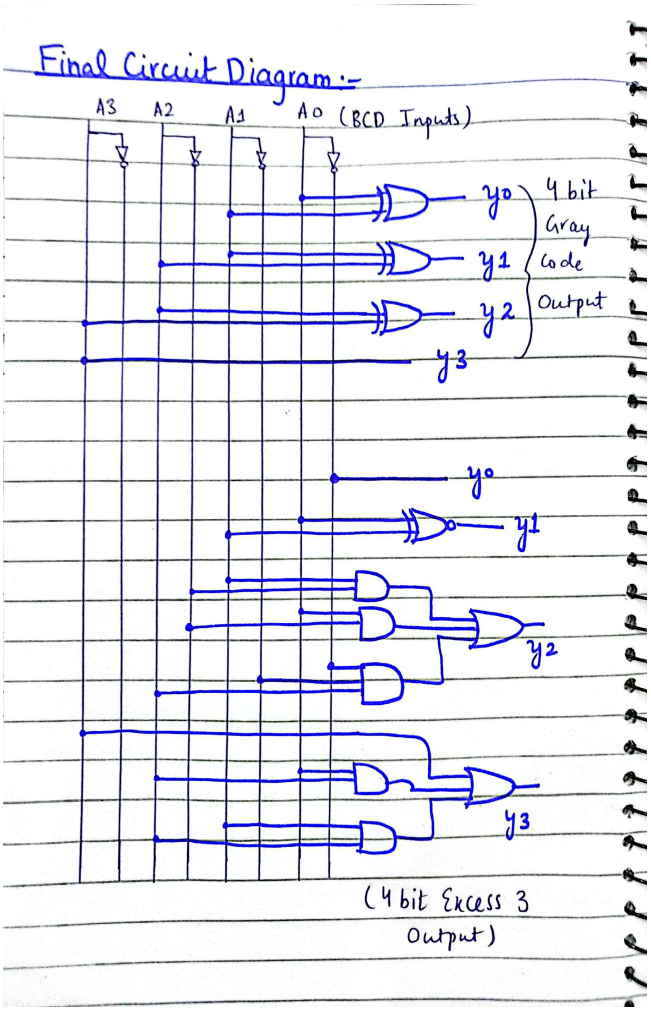


Figure 4: Combined Gray and excess-3 reference drawing. Source: GitHub user Mmamoonn. The lower half is the excess-3 circuit.

Circuit drawing source:

https://github.com/Mmamoonn/code-conversion-system/blob/main/hardware/circuit-diagrams/final_circuit_diagram.jpeg

Structural excess-3 VHDL:

https://github.com/Mmamoonn/code-conversion-system/blob/main/vhdl/src/bcdtoex3.vhd

Gray source requiring care:

https://github.com/Mmamoonn/code-conversion-system/blob/main/vhdl/src/bcdtogray.vhd

Drawing/code discrepancy: the repository's Gray code uses OR for g_2, whereas the drawing shows XOR. OR agrees with this XOR on valid BCD inputs 0–9 but fails for binary inputs 12–15. Use the preceding three-XOR circuit for our 0–15 Gray tests. Excess-3 tests cover valid BCD inputs 0–9: display the input through a BCD decoder on HEX2 and the converted code through a Hex decoder on HEX3.